

# CubeDynamics manuscript: citation map

Working draft - citation locations and source families

Citation markers are editorial annotations, not final references. They identify claims that should be supported during the reference audit; implementation statements about CubeDynamics generally need only the archived software release rather than repeated citations.

## Abstract

Environmental data infrastructure has become remarkably successful at making observations accessible. Climate records, satellite imagery, disturbance histories, hydrological measurements, and other environmental data that once required substantial effort to acquire can increasingly be queried directly from public archives and cloud-hosted collections.

**CITE: environmental cloud/data infrastructure, STAC/cloud-native environmental data, and major public archives.**

Yet easier access has not made multi-source environmental analysis proportionally easier to understand. As analyses incorporate more observations, scientific decisions about spatial and temporal support, source selection, aggregation, alignment, units, missing data, and operation order become interleaved with increasingly complex computational workflows.

**CITE: environmental data harmonization and interoperability literature.**

We present CubeDynamics, an open-source Python package that addresses this problem by expressing analysis as a composable grammar over spatiotemporal environmental cubes. CubeDynamics provides a deliberately small analytical structure in which environmental observations are represented as cubes, scientific transformations are expressed as verbs, and explicit pipes preserve the order in which those transformations occur. The package operates over the existing scientific Python ecosystem rather than replacing it, using xarray-compatible objects, Dask-backed lazy computation, geospatial libraries, remote catalogs, and maintained environmental-data adapters.

**CITE: xarray, Dask, and STAC/pystac-client software references.**

This abstraction is paired with a layered validation architecture that distinguishes deterministic software behavior, live-source contracts, spatial and coordinate assumptions, lazy execution, source quality assurance, analytical decision checks, executable documentation, and provenance. As an environmental analysis becomes easier to express, the software assumes greater responsibility for demonstrating what data were retrieved, which transformations were performed, and which scientific decisions remain with the analyst.

## 1. Introduction

Environmental data science has made remarkable progress at making observations accessible. Measurements that once required locating an archive, requesting files, downloading large collections, and constructing a local processing pipeline can increasingly be discovered through catalogs and queried directly from cloud-hosted collections. Decades of gridded climate observations can be subset to a study region without downloading a continental archive. Satellite imagery can be discovered through standardized catalogs and processed close to where it is stored. Environmental observations distributed among agencies and research programs can increasingly be reached through programmatic interfaces.

**CITE: STAC; cloud-optimized geospatial formats; Earth-observation cloud platforms; representative NASA, NOAA, USGS, or Microsoft Planetary Computer infrastructure.**

That success has exposed a different limitation. Retrieving ten environmental datasets has become substantially easier; reasoning clearly about the transformations required to make those ten datasets answer one scientific question has not. A temperature product may be daily and kilometer-scale, a vegetation product may be observed every several days at tens of meters, a fire record may consist of irregular polygons, and a stream gauge may represent a point observation integrating an upstream watershed. Before these observations can contribute to a common analysis, the scientist must decide what constitutes the relevant place, time period, spatial support, temporal support, variable, unit, aggregation, and comparison.

**CITE: environmental data integration/harmonization literature and literature on scale, support, and interoperability.**

As the number of observations grows, these scientific decisions become increasingly entangled with implementation. A single environmental workflow may contain catalog queries, authentication, URLs, filenames, coordinate transformations, array indexing, spatial intersections, temporal grouping, interpolation, masking, lazy computation, joins, intermediate objects, plotting instructions, and export code. Mature scientific Python libraries provide excellent tools for performing these tasks, but their flexibility necessarily exposes implementation details.

**CITE: grouped scientific Python ecosystem references if desired.**

General-purpose libraries such as xarray, Dask, rasterio, rioxarray, GeoPandas, Shapely, and STAC clients already provide much of the computational machinery required for environmental analysis. CubeDynamics does not attempt to replace them.

**CITE: formal software citations for these major dependencies.**

Abstraction offers an obvious solution, but it creates an equally important risk. A concise function can hide dozens of operations. A human-readable environmental variable can conceal the service, product, version, units, coordinate conventions, and preprocessing choices from which it was constructed. Scientific software therefore faces a tension: reducing incidental complexity can make an analysis easier to understand, but excessive abstraction can make scientifically consequential decisions harder to see.

**CITE: scientific workflow transparency, provenance, reproducibility, and analytical-decision literature.**

CubeDynamics was developed around the premise that these two goals do not have to be opposed. We organize this principle around three related responsibilities: expression, verification, and explanation. The current implementation realizes this approach through a deliberately small grammar over spatiotemporal cubes, environmental-data adapters, and Dask-backed execution.

**CITE: CubeDynamics archived release; Dask for lazy execution.**

## 2. A grammar for environmental analysis

The stable center of CubeDynamics is intentionally small because a grammar is useful only when its rules remain predictable. The package begins with an xarray-compatible environmental object, wraps that object with pipe, and applies verbs through the | operator.

**CITE: CubeDynamics archived release/documentation.**

```
result = (\n pipe(cube)\n | v.anomaly(dim="time")\n | v.variance(dim="time",\n keep_dim=False)\n).unwrap()
```

The significance of this expression is not that it saves several lines of Python. The important feature is that the analytical sequence is locally visible. Environmental operations are frequently noncommutative: calculating an anomaly before spatial averaging need not answer the same question as calculating a regional mean before determining its anomaly.

**CITE OPTIONAL: mathematical/statistical literature on aggregation order, or demonstrate the distinction empirically in the manuscript instead.**

CubeDynamics treats the spatiotemporal cube as a central contract. The documentation introduces cubes from NumPy arrays, tidy observations, and xarray Datasets, converging toward explicitly labeled spatial and temporal dimensions.

**CITE: xarray data model and environmental data-cube literature.**

Keeping space and time within the same labeled object reduces fragmentation among tabular, raster, temporal, and visualization stages.

**CITE: environmental data-cube and analysis-ready-data literature.**

CubeDynamics supports lazy Dask-backed xarray objects so that the same pipe can describe both an eager demonstration and a larger deferred computation.

**CITE: Dask and xarray-Dask integration; optionally cloud-native/lazy array-computation literature.**

The resulting architecture is deliberately layered: xarray and NumPy provide array semantics, Dask provides lazy execution, geospatial libraries provide spatial operations, remote catalogs provide observations, and CubeDynamics provides semantic composition.

**CITE: principal software references.**

### 3. Environmental observations enter the grammar without becoming interchangeable

The current documentation includes maintained pathways for environmental products such as gridMET, PRISM, Sentinel-2 NDVI, Landsat 8 through Microsoft Planetary Computer, and FIRED.

**CITE EACH DATA PRODUCT: gridMET; PRISM; Sentinel-2; Landsat 8; Microsoft Planetary Computer; FIRED.**

Two products described as temperature observations may differ in spatial resolution, interpolation methodology, temporal representation, units, coverage, and update schedule. Syntactic interoperability is therefore not evidence that the products are scientifically equivalent.

**CITE: product papers/documentation for the climate products used in the worked comparison.**

CubeDynamics begins harmonization by standardizing the interface through which spatial, temporal, and semantic properties can be inspected and transformed rather than asserting equivalence among observations.

**CITE: FAIR/interoperability/harmonization literature.**

A user should not need a fundamentally different analytical vocabulary because one observation arrives through NCSS, another through STAC, another from a cloud-optimized raster, and another from a local cache.

**CITE: NCSS, STAC, and COG standards/specifications where retained in the text.**

Environmental archives are living infrastructure: providers migrate endpoints, update products, alter schemas, and release new versions. Source configuration and serving history must therefore be maintainable resources.

**CITE OPTIONAL: persistent identifiers, data versioning, and provenance literature.**

### 4. Making abstraction accountable

Abstraction changes what scientific software must test. A short analytical expression can invoke numerical behavior, metadata preservation, remote-source resolution, lazy execution, spatial contracts, and documentation behavior.

**CITE: scientific software testing, reproducibility, and workflow-validation literature.**

CubeDynamics separates deterministic offline tests from integration and online tests so that core regressions can be distinguished from remote-provider failures.

**CITE: CubeDynamics release/CI only; no external citation required.**

A Dask-backed pipeline can be numerically correct on a small fixture while accidentally forcing an entire remote cube into memory. CubeDynamics therefore treats laziness as behavior worth testing.

**CITE: Dask/lazy-execution reference.**

Spatial data can retain plausible values while losing coordinate orientation or CRS information.

**CITE OPTIONAL: geospatial CRS/raster standards or rasterio/rioxarray references.**

Source quality assurance asks whether the requested variable, dimensions, coordinates, temporal coverage, units, missingness, and numerical ranges remain scientifically credible.

**CITE: environmental data-quality/QA literature; source documentation for expected units/ranges.**

Treating vignettes as executable artifacts prevents documentation from silently drifting away from the installed API.

**CITE: executable documentation, literate programming, reproducible notebook/workflow literature.**

Taken together, these layers reflect a simple principle: the shorter the analytical expression becomes, the stronger the evidence underneath it must become.

### 5. Analytical order is part of the result

Software verification alone cannot establish that a valid sequence of operations answers the scientific question the analyst intended.

**CITE OPTIONAL: analytical flexibility, reproducible workflows, or researcher-degrees-of-freedom literature.**

```
pipe(cube) | v.anomaly(dim="time") | v.mean(dim=("y", "x"))
```

Reversing these operations can produce a scientifically different quantity depending on baseline definition, weighting, missing observations, spatial heterogeneity, and nonlinear transformations.

**CITE OPTIONAL: formal statistical support, or provide an empirical demonstration in a figure.**

Provenance connects source identity, transformations, dimensions, units, spatial and temporal context, and operation history.

**CITE: W3C PROV and/or scientific provenance/reproducible workflow literature.**

This inspectable representation may also be useful in collaborative and AI-assisted scientific workflows.

**CITE OPTIONAL: agentic science / AI-for-science literature only if this argument remains in the final paper.**

## 6. CubeDynamics in practice

The current CubeDynamics documentation includes gridded climate analyses, remotely sensed vegetation, event-oriented transformations, synchrony analyses, and Fire VASE workflows.

**CITE: CubeDynamics release/documentation; external source papers for datasets used in final examples.**

Climate examples use products such as gridMET and PRISM.

**CITE: gridMET and PRISM.**

Remote-sensing examples use Sentinel-2 NDVI or Landsat observations accessed through cloud catalogs.

**CITE: Sentinel-2, Landsat, STAC, and Microsoft Planetary Computer as applicable.**

Fire VASE uses evolving wildfire perimeters and environmental covariates.

**CITE: FIRED and any Fire VASE publication if available; otherwise cite CubeDynamics/software functionality only.**

Synchrony analyses use specialized state, event, and coupling concepts.

**CITE: relevant ecological synchrony literature and any published method actually implemented.**

## 7. Discussion

CubeDynamics is complementary to xarray, Dask, geospatial libraries, and remote catalog standards rather than a replacement for them.

**CITE: xarray, Dask, rasterio/rioxarray, GeoPandas/Shapely, STAC.**

A concise grammar can become less scientifically transparent without safeguards that preserve provenance and analytical decisions.

**CITE: provenance, reproducibility, and scientific workflow transparency literature.**

The longer-term motivation is that environmental analyses are incorporating more observations and transformations.

**CITE OPTIONAL: broad environmental synthesis/data-integration literature if this sentence remains.**

The present manuscript does not test a 100-dataset synthesis; it evaluates the implemented grammar and validation architecture.

## 8. Conclusions and availability

Environmental data infrastructure has made observations increasingly accessible, while multi-source synthesis still requires explicit analytical decisions.

**CITE: introduction sources need not necessarily be repeated here.**

CubeDynamics uses labeled spatiotemporal cubes, explicit verbs and pipes, maintained adapters, and Dask-backed execution.

**CITE: CubeDynamics archived release; Dask where needed.**

CubeDynamics is developed openly in the CU-ESIL cubedynamics repository and distributed under the MIT License.

**CITE: final archived software release/DOI in Code and Data Availability.**

## Reference families to assemble

1. Environmental data access and cloud infrastructure: STAC, cloud-native geospatial formats, major Earth-observation/cloud platforms.
2. Scientific Python infrastructure: xarray, Dask, rasterio/rioxarray, GeoPandas, Shapely, NumPy as appropriate.
3. Environmental data harmonization, interoperability, data cubes, and analysis-ready data.
4. Reproducibility, provenance, executable workflows, and scientific software validation.
5. Environmental products used by the release/examples: gridMET, PRISM, Sentinel-2, Landsat, FIRED, and any others retained.
6. Domain methods: synchrony, fire/event methods, and AI-assisted science only where the manuscript actually makes those claims.